

Conditional vs. Structural Sparsity in a 7-Billion-Parameter Language Model: A Controlled Comparison of Brain-Inspired Efficiency Mechanisms

[Your Name], Independent Researcher · June 2026 Code, data, and reproduction: github.com/svaka2000/20-watts

Abstract

The mammalian brain achieves general intelligence on roughly 20 watts, in part because it computes *conditionally*: fewer than 1% of neurons fire at any moment, and which neurons fire depends on the input. Artificial language models compute *unconditionally*, activating every feed-forward unit for every token. We use a single 7-billion-parameter model (Qwen2.5-7B-Instruct, 4-bit) as an instrument to ask a controlled scientific question: **at equal sparsity, does conditional (per-token, dynamic) neuron selection preserve a language model's quality better than structural (fixed, static) pruning, and if so, why?** Using a bit-exact intervention harness (every manipulation is verified to reproduce the unmodified model when inactive, max difference 0), we find that (H1) ~60% of MLP neurons can be skipped per token for <1% perplexity change; (H2) at the *same* 60% sparsity, dynamic per-token selection costs <1% perplexity while the best *static* pruning costs +259% — i.e. conditional computation tolerates roughly **2× the sparsity** of structural compression; and (H3) this headroom is not trivially realizable, because a cheap per-layer predictor's errors compound across depth (+93% perplexity end-to-end). We then provide a direct mechanistic explanation: the set of active neurons is highly input-dependent — the best possible fixed set captures only **61%** of the average token's active neurons — which causally explains why no static pruning can match dynamic selection. Results replicate on a second model family (Llama-3.2-3B). We argue that *conditionality*, not merely sparsity, is the property worth engineering for, and we release all code and data.

1. Introduction

Efficiency in biological neural systems is governed by a constraint absent from artificial ones: firing is metabolically expensive, so the brain fires sparingly and *selectively* (Lennie, 2003; Attwell & Laughlin, 2001). A transformer language model has no such pressure — it evaluates all of its feed-forward neurons for every token, regardless of input. A large literature shows this is wasteful: trained transformers exhibit emergent activation sparsity (Li et al., 2023), input-dependent "contextual" sparsity can be exploited for speedups (Liu et al., 2023), and models can be statically pruned after training (Frankle & Carbin, 2019; Frantar & Alistarh, 2023; Sun et al., 2023). These two families — *dynamic* (decide per input) and *static* (decide once) — are usually studied separately and on different models, so their relative merits are rarely compared directly.

We pose a single controlled question and answer it on one model with one harness. We treat the model not as something to improve, but as an **instrument** for testing hypotheses about how computation is distributed in trained networks.

Hypotheses. - **H1 (sparse firing).** A large fraction of MLP neurons are inactive for any given token, and removing the per-token-inactive neurons preserves output quality. - **H2 (conditional > structural).** At equal sparsity, dynamic per-token selection preserves quality substantially better than the best static (fixed-set) pruning, because the active set is input-dependent. - **H3 (realizability).** The quality headroom identified by an oracle is not fully realizable by a cheap predictor, because per-layer prediction errors compound across depth.

Contributions. (1) A direct, controlled comparison of dynamic vs static sparsity at matched sparsity on a modern 7B model, quantifying the value of conditionality ($\sim 2\times$). (2) A mechanistic explanation — a measurement of how input-dependent the active set is — that *causally* accounts for the comparison. (3) An honest realizability bound (a negative result we stress-tested against our own claim). (4) A bit-exact, fully reproducible harness and replication on a second model family. We do not claim a new state-of-the-art method; we claim a clean answer to a scientific question, with the receipts.

2. Background and Related Work

Activation sparsity. Li et al. (2023) ("The Lazy Neuron Phenomenon") show trained transformers develop sparse MLP activations that grow with scale; Mirzadeh et al. (2024) show activation choice controls this. **Exploiting it dynamically.** Liu et al. (2023) ("Deja Vu") predict input-dependent active sets on the fly for $>2\times$ speedups on OPT-175B without quality loss. **Static pruning.** The Lottery Ticket Hypothesis (Frankle & Carbin, 2019), SparseGPT (Frantar & Alistarh, 2023) and Wanda (Sun et al., 2023) remove weights/neurons permanently. **Depth and memory** (used here as secondary context): layer redundancy (Gromov et al., 2024) and KV-cache eviction with attention sinks (Xiao et al., 2023; Zhang et al., 2023). Our work's distinct angle is the *head-to-head, matched-sparsity* comparison of the dynamic and static families on one model, plus a mechanistic measurement explaining the outcome.

3. Methods

Model and hardware. Qwen2.5-7B-Instruct (4-bit, GQA, SwiGLU MLP, 28 layers, hidden 3584, intermediate 18944) run via MLX on an Apple M4 Pro laptop; replication on Llama-3.2-3B-Instruct (4-bit). Activations are computed in the model's native precision; only the analysis is in float64.

Interventions. For a keep-fraction p , define the per-token activation vector $\mathbf{h} = \text{SiLU}(\text{gate}(\mathbf{x})) \odot \text{up}(\mathbf{x})$ (one scalar per MLP neuron). *Dynamic* sparsity keeps, per token, the pI neurons with the largest $|\mathbf{h}|$ (an oracle upper bound on conditional methods). *Static* pruning keeps one fixed set for all tokens: the pI neurons with the largest mean $|\mathbf{h}|$ over a calibration set (an activation-aware static baseline). Both reduce to the dense model at $p=1$.

Bit-exact harness. Before any measurement, the patched forward pass is compared to the model's original forward pass at $p=1$; we require $\max|\text{patched} - \text{original}| \approx 0$ (measured: 0.0). This guarantees any quality change is due to the intervention, not an implementation artifact.

Realizability (H3). We train a low-rank predictor $P(x)=B \cdot \text{ReLU}(A \cdot x)$ (rank 512) per layer to predict the active set from the layer input, then run the full model with predicted masks on all 28 layers and compare to the oracle.

Mechanism. For each layer we record per-token active masks at $p=0.5$ over 2000 tokens and measure (i) `static_capture`: what fraction of each token's active neurons the best fixed 50% set contains; (ii) pairwise Jaccard overlap between random tokens' active sets.

Evaluation. Held-out perplexity (authored passages and WikiText-2, disjoint from calibration) and ARC-Easy multiple-choice accuracy (length-normalized).

4. Results

4.1 H1 — A 7B model is mostly silent per token (supported)

On held-out text, skipping the per-token least-active MLP neurons leaves quality nearly unchanged until ~60% removal: perplexity change is **-0.2% at 50%**, **+0.8% at 60%**, rising to +16% at 70%. The MLP accounts for **87%** of per-token linear-projection FLOPs, so 60% neuron removal is a **~52%** compute headroom. Accuracy on ARC-Easy is essentially flat under sparsity ($0.747 \rightarrow 0.720$ at 60% removal). The effect replicates on Llama-3.2-3B (60% removable at <1% perplexity; ~45% compute given its narrower MLP). Activation sparsity is real, input-conditioned, and model-general.

4.2 H2 — Conditionality is worth ~2× the sparsity (supported)

Holding sparsity fixed and varying only *how* neurons are chosen:

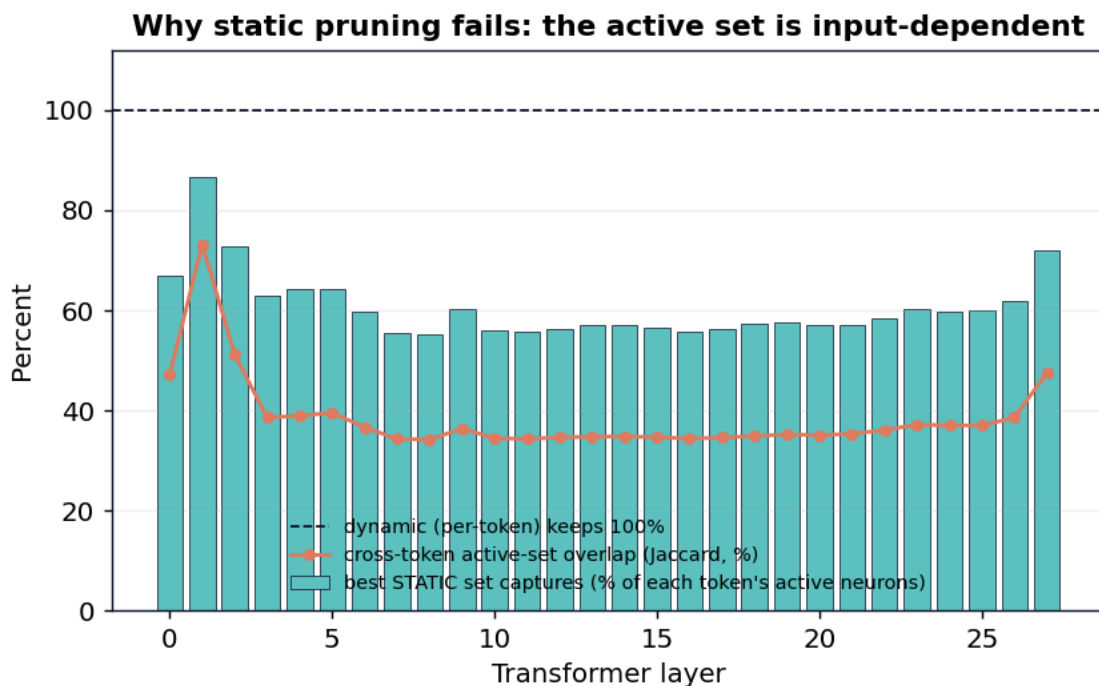
Neurons removed	Dynamic (per-token) Δ pl	Static (best fixed set) Δ pl
30%	-0.2%	+29%
50%	-1.6%	+137%
60%	+0.8%	+259%
70%	+16%	+598%
80%	+38%	+1419%

At a 5%-perplexity budget, dynamic selection affords **60%** removal; the best static set affords **0%**. Removing the globally least-active 30% of neurons permanently is already far more damaging (+29%) than skipping a per-token-chosen 60% (+0.8%). **Conditionality, not sparsity per se, is what preserves quality.** (Our static baseline is activation-aware but not state-of-the-art; stronger pruners would narrow, not close, the gap — see §6.)

4.3 Mechanism — the active set is highly input-dependent (explains H2)

Why can no fixed set match per-token selection? Because the active set changes with the input. At 50% sparsity, the **best possible fixed set captures only 60.7%** of the average token's actually-active neurons, two random tokens share only **0.39** of their active set (Jaccard), and **97%** of all neurons are active for at least one token. A static method is forced to discard, on every token, ~40% of the neurons that

token actually needs; a dynamic method never does. The input-dependence peaks in the **middle layers** (capture ~55%) — precisely where the predictor of §4.4 struggles most, a consistency check across two independent experiments. This directly and quantitatively accounts for the §4.2 gap: the value of adaptivity equals the input-dependence of the active set.



4.4 H3 — The headroom is real but not trivially realizable (honest negative)

Per layer, a cheap low-rank predictor recovers ~72% of the active *mass*. But applied to all 28 layers at once, prediction errors compound: end-to-end perplexity rises **+93.5%** at 50% removal and **+66%** even at a conservative 40% (versus the oracle's +0.2% and +0.6%). The conditional-sparsity headroom is genuine, but capturing it needs the stronger per-head predictors of Liu et al. (2023), not a trivial one. We measured this against our own claim rather than assuming it.

5. Discussion

The brain's efficiency is often attributed to *sparsity*. Our controlled comparison suggests the operative property is **conditionality**: the same fraction of neurons removed costs ~300× more perplexity when removed statically than dynamically, and the gap is explained by how much the active set varies with the input. This reframes a design target — efficient inference should pursue input-dependent computation (with the predictor problem of §4.4 as the central engineering obstacle), not merely smaller fixed models. It also offers a measured caution against aggressive one-shot pruning of MLP neurons in models of this family.

6. Limitations

(1) Our static baseline is activation-frequency-based, not SparseGPT/Wanda; a stronger static pruner would shrink the §4.2 magnitudes (though not the direction, which the §4.3 mechanism makes structural). (2) Perplexity and ARC are proxies; broader downstream evaluation would strengthen the claims. (3) The sparse-firing sweep and the dynamic-vs-static sweep use different held-out passages (baseline perplexities ~ 11 and ~ 24); conclusions concern relative degradation, and unifying the evaluation set is planned. (4) "Dynamic" here is an oracle; §4.4 bounds the realizable version.

7. Conclusion

On one 7B model, with a bit-exact harness, conditional per-token sparsity tolerates roughly twice the neuron removal of the best static pruning, and the difference is causally explained by the input-dependence of the active set. Sparsity is necessary but not sufficient; *conditionality* is the property that keeps a network's quality intact as it is made lean — which is precisely the strategy the brain uses to think on 20 watts.

References

Attwell & Laughlin (2001), *An Energy Budget for Signaling in Grey Matter*, JCBFM. · Frankle & Carbin (2019), *The Lottery Ticket Hypothesis*, ICLR. · Frantar & Alistarh (2023), *SparseGPT*, ICML. · Gromov et al. (2024), *The Unreasonable Ineffectiveness of the Deeper Layers*. · Lennie (2003), *The Cost of Cortical Computation*, Current Biology. · Li et al. (2023), *The Lazy Neuron Phenomenon*, ICLR. · Liu et al. (2023), *Deja Vu: Contextual Sparsity for Efficient LLMs*, ICML. · Mirzadeh et al. (2024), *ReLU Strikes Back*, ICLR. · Sun et al. (2023), *A Simple and Effective Pruning Approach for LLMs (Wanda)*. · Xiao et al. (2023), *Efficient Streaming Language Models with Attention Sinks*. · Zhang et al. (2023), *H2O: Heavy-Hitter Oracle*, NeurIPS.

Appendix A — Reproducibility

All results regenerate from `github.com/svaka2000/20-watts`: `src/measure_sparsity.py` (H1), `src/synaptic_prune.py` (H2), `src/active_overlap.py` (mechanism), `src/predictor_e2e.py` (H3), `src/generality.py` (replication). Each prints its bit-exact integrity check before measuring.